added (so there are no false-negatives, i.e. every time it answers false we are sure the answer is correct).

Bloom filters too are a tradeoff between memory and accuracy. The deterministic version of a Bloom filter is a *hashset*.

4.9 Performance Analysis

Before starting on Bloom filter analysis, I suggest a deep dive into metrics for classification algorithms in appendix F.

We have seen how and why Bloom filters work, now let's check how efficient they are. First, we'll examine the running time for the most important operations provided by a Bloom filter. Next, in section 4.10, we will tackle the challenge of predicting the precision of method `contains`, given a Bloom filter with a certain structure (in particular, its size and the number of hash functions used will matter).

4.9.1 Running Time

We have already hinted at the fact that Bloom filters can store and lookup a key in constant time. Technically this is only true for constant-length inputs: here we will examine the most generic case, when keys stored are strings of arbitrary length[18].